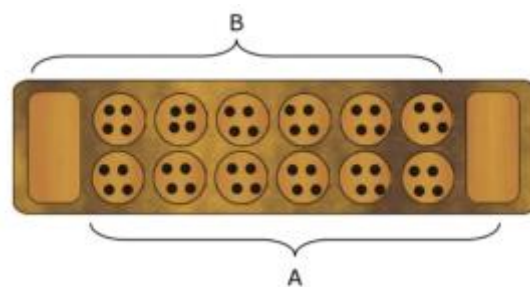


Jogo "Ouri" em Linguagem C



Universidade: Colégio Luís António Verney
(Universidade de Évora)

Curso: Engenharia Informática

Tiago Hermitério nº 59111
Pedro Soares nº 59115
Francisco Rodrigues nº 59119

Índice

Introdução	3
Estrutura do Jogo	3
Regras do Jogo.....	3
Objetivo da Implementação.....	3
Estrutura do Código	3
Abordagem ao Problema.....	4
Dificuldades Encontradas	4
Soluções Adotadas.....	4
Instruções.....	4
Funções Criadas	5
* `main(int argc, char *argv[])`	5
* `inicializarTabuleiroFromFile(int tabuleiro[], char *nomeFile)`	6
* `inicializarTabuleiro(int tabuleiro[], int pedrasPorBuraco)`	7
* `mostrarTabuleiro(int tabuleiro[])`	7
* `jogarJogo(int tabuleiro[], int modoDeJogo)`	7
* `fazerMovimento(int tabuleiro[], int jogadorAtual, int buracoSelecionado)`	8
* `obterBuracoOposto(int buraco)`	9
* `verificarVencedor(int tabuleiro[], int jogadorAtual)`	9
* `obterMovimentoComputador(int tabuleiro[])`	10
Melhorias na Estrutura do Código.....	11
Conclusão	11

Introdução

Este estudo propõe uma análise abrangente sobre um modelo dinâmico em linguagem C, cujo foco incide na criação de um jogo "Ouri". O sistema em questão apresenta-se como um desafio estratégico envolvendo dois participantes, composto por um tabuleiro com 14 buracos e 48 pedras. O objetivo é coletar mais pedras do que o adversário, vencendo quem atingir 25 ou mais pedras. Este relatório discute a implementação do jogo em linguagem C.

Estrutura do Jogo

O tabuleiro possui duas filas de seis buracos, denominadas casas, e dois buracos maiores nas extremidades, chamados depósitos. Os jogadores sentam-se frente a frente, com as suas casas numeradas de 1 a 6. Cada jogador possui um depósito à sua direita. O jogo inicia com o jogador A.

Regras do Jogo

As regras incluem movimentos de distribuição de pedras, capturas quando a última pedra é colocada em uma casa adversária com 2 ou 3 pedras, e reposição de pedras quando um jogador fica sem pedras ao seu lado. A partida termina quando um jogador atinge 25+ ou quando o adversário não pode mais jogar.

Objetivo da Implementação

O objetivo da implementação em linguagem C é criar um programa que simula o jogo "Ouri", oferecendo modos de jogador contra jogador ou jogador contra computador. O programa interage com os jogadores, exibindo o estado do tabuleiro e permitindo movimentos válidos.

Estrutura do Código

A codificação foi concebida através da segmentação em dois ficheiros distintos: um de cabeçalho (.h) e outro de código fonte (.c), garantindo modularidade e prevenindo inclusões múltiplas. O ficheiro de cabeçalho (ouri.h) encerra estruturas que encapsulam a configuração do tabuleiro, enquanto funções pertinentes à inicialização, exibição, manobras, verificação de condições de vitória e manipulação de ficheiros são declaradas. As funções foram implementadas no arquivo .c correspondente, seguindo a lógica do jogo.

Abordagem ao Problema

A implementação do jogo "Ouri" em C foi abordada com ênfase na modularidade, buscando clareza e facilidade de manutenção. Uma estrutura foi definida para representar o estado do tabuleiro, incluindo casas, depósitos e informações sobre o jogador atual. As regras foram traduzidas em funções específicas para facilitar a compreensão e execução do código.

Dificuldades Encontradas

Durante o desenvolvimento, a complexidade das regras, sobretudo as condições de captura, apresentou desafios. Garantir que as funções estivessem em conformidade com as condições específicas de movimentação e captura exigiu uma análise minuciosa das regras. A implementação do modo jogador contra computador demandou considerações adicionais para criar um adversário virtual eficaz.

Soluções Adotadas

Para contornar os desafios, uma abordagem iterativa foi adotada. Cada função foi implementada passo a passo, com testes regulares para assegurar o seu funcionamento adequado. A implementação seguiu uma lógica sequencial, começando pela inicialização do tabuleiro, exibição do estado, execução de movimentos e verificação das condições de vitória. A modularidade foi uma prioridade, simplificando a compreensão e manutenção do código.

Instruções

No terminal (Linux), escreva `cd "diretório onde se encontra o ficheiro"`, de seguida escreva `gcc -o "ourí".c "ourí.h"` e depois faça `./ourí` para abrir o jogo normalmente ou `./ourí nomedoficheiro.txt` para carregar o tabuleiro do jogo a partir de um ficheiro txt.

Funções Criadas

* ``main(int argc, char *argv[])``

- Propósito: Definir variáveis, verificar se o tabuleiro é iniciado via ficheiro ou normalmente, escolha do modo de jogo.
- Argumentos: Contador de argumentos, vector de argumentos.
- Valor de Retorno: Int.

```
int main(int argc, char *argv[])
{
    int tabuleiro[TAM_TABULEIRO];
    int pedrasPorBuraco = 4; // Definir pedras iniciais por buraco
    int modoDeJogo;

    if (argc == 2){
        inicializarTabuleiro(tabuleiro, pedrasPorBuraco);
        inicializarTabuleiroFromFile(tabuleiro, argv[1]);
    }
    else{
        inicializarTabuleiro(tabuleiro, pedrasPorBuraco);
    }

    printf("Bem-vindo ao jogo de Ouri!\n");
    printf("Escolha o modo de jogo:\n");
    printf("1. Jogador vs Jogador\n");
    printf("2. Jogador vs Computador\n");
    printf("Opção: ");
    scanf("%d", &modoDeJogo);

    if (modoDeJogo != 1 && modoDeJogo != 2)
    {
        printf("Opção inválida. Escolha 1 ou 2.\n");
        return 1;
    }

    mostrarTabuleiro(tabuleiro);
    jogarJogo(tabuleiro, modoDeJogo - 1); // Ajustar para 0 ou 1 para representar o modo de jogo

    return 0;
}
```

*** `inicializarTabuleiroFromFile(int tabuleiro[], char \*nomeFile)`**

- Propósito: Inicializa o tabuleiro com os valores guardados num ficheiro txt.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo, string que representa o nome do ficheiro que se quer abrir.
- Valor de Retorno: Int.

```
int inicializarTabuleiroFromFile(int tabuleiro[], char *nomeFile)
{
    FILE *file = fopen(nomeFile, "r");

    if (file == NULL)
    {
        // Tentar abrir o ficheiro removendo a extensão .txt
        char nomeArquivoSemTxt[50];
        strcpy(nomeArquivoSemTxt, nomeFile);
        char *p = strrchr(nomeArquivoSemTxt, '.'); // Encontrar o último ponto na string

        if (p != NULL)
        {
            *p = '\0'; // Remover a extensão .txt
            file = fopen(nomeArquivoSemTxt, "r"); // Tentar abrir o arquivo novamente sem a extensão
        }

        if (file == NULL)
        {
            perror("Erro ao abrir o ficheiro\n");
        }
    }

    fscanf(file, "%d", &tabuleiro[13]);
    for (int i = 12; i >= 7; --i)
    {
        fscanf(file, "%d", &tabuleiro[i]);
    }
    fscanf(file, "%d", &tabuleiro[6]);
    for (int i = 0; i < 6; ++i)
    {
        fscanf(file, "%d", &tabuleiro[i]);
    }

    fclose(file);
}
```

*** `inicializarTabuleiro(int tabuleiro[], int pedrasPorBuraco)`**

- Propósito: Inicializa o tabuleiro conforme as regras do jogo.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo, o número inicial de pedras por buraco.
- Valor de Retorno: Void.

```
void inicializarTabuleiro(int tabuleiro[], int pedrasPorBuraco)
{
    for (int i = 0; i < TAM_TABULEIRO; ++i)
    {
        if (i != 6 && i != 13)
        { // Excluir os armazéns
            tabuleiro[i] = pedrasPorBuraco;
        }
        else
        {
            tabuleiro[i] = 0; // Os armazéns começam vazios
        }
    }
}
```

*** `mostrarTabuleiro(int tabuleiro[])`**

- Propósito: Exibe o tabuleiro na consola.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo.
- Valor de Retorno: Void.

```
void mostrarTabuleiro(int tabuleiro[])
{
    printf("\n Jogador B (6-1)\n");
    printf(" |---|---|---|---|---|---|---|---|\n");
    printf(" | %2d| %2d| %2d| %2d| %2d| %2d| | \n", tabuleiro[12], tabuleiro[11], tabuleiro[10], tabuleiro[9], tabuleiro[8], tabuleiro[7]);

    printf(" |%2d |-----| %2d | \n", tabuleiro[13], tabuleiro[6]);

    printf(" | %2d| %2d| %2d| %2d| %2d| %2d| | \n", tabuleiro[0], tabuleiro[1], tabuleiro[2], tabuleiro[3], tabuleiro[4], tabuleiro[5]);
    printf(" |---|---|---|---|---|---|---|---|\n");
    printf(" Jogador A (1-6)\n");
}
```

*** `jogarJogo(int tabuleiro[], int modoDeJogo)`**

- Propósito: Orquestra o fluxo do jogo "Ouri", gere turnos, movimentos e atualizações do estado do jogo.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo, modo de jogo que irá determinar se o adversário é outro jogador (Player B) ou o computador.
- Valor de Retorno: Void.

*** `fazerMovimento(int tabuleiro[], int jogadorAtual, int buracoSelecionado)`**

- Propósito: Atualizar o tabuleiro do jogo com base no movimento do jogador, lidar com o posicionamentos das pedras, capturas e atualizar o estado do jogo de acordo.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo, o jogador que está atualmente a fazer a jogada (Pode ser Jogador ou Computador, onde Jogador representa um jogador humano e Computador representa o oponente computado), buraco selecionado de momento pelo jogador.
- Valor de Retorno: Int.

```
int fazerMovimento(int tabuleiro[], int jogadorAtual, int buracoSelecionado)
{
    int pedras = tabuleiro[buracoSelecionado];
    tabuleiro[buracoSelecionado] = 0;

    int posicao = buracoSelecionado;

    while (pedras > 0)
    {
        posicao = (posicao + 1) % TAM_TABULEIRO;

        if (jogadorAtual == PLAYER && posicao == 13)
        { // Saltar depósito do oponente
            continue;
        }
        else if (jogadorAtual == COMPUTER && posicao == 6)
        { // Saltar depósito do oponente
            continue;
        }

        if (posicao == jogadorAtual * 7 + 6)
        {
            // Se for o depósito do jogador atual, pular para o próximo buraco
            posicao = (posicao + 1) % TAM_TABULEIRO;
        }

        tabuleiro[posicao]++;
        pedras--;

        if (pedras == 0 && posicao != jogadorAtual * 7 + 6 && tabuleiro[posicao] == 1 && ((posicao >= 0 && posicao <= 5 && jogadorAtual == PLAYER) || (posicao >= 6 && posicao <= 12 && jogadorAtual == COMPUTER)))
        {
            int buracoOposto = obterBuracoOposto(posicao);
            tabuleiro[jogadorAtual * 7 + 6] += tabuleiro[buracoOposto];
            tabuleiro[buracoOposto] = 0;
        }
    }

    return posicao;
}
```


*** `obterBuracoOposto(int buraco)`**

- Propósito: projetado para encontrar o buraco oposto a um determinado buraco no tabuleiro de jogo, auxilia na lógica do jogo relacionada à captura e movimento de pedras no jogo "Ouri".
- Argumentos: Index do buraco atual para o qual deseja encontrar o buraco oposto.
- Valor de Retorno: Int.

```
int obterBuracoOposto(int buraco)
{
    return 12 - buraco;
}
```

*** `verificarVencedor(int tabuleiro[], int jogadorAtual)`**

- Propósito: Verificar a existência de um vencedor.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo, o jogador que está atualmente a fazer a jogada (Pode ser Jogador ou Computador, onde Jogador representa um jogador humano e Computador representa o oponente computado).
- Valor de Retorno: Int.

```
int verificarVencedor(int tabuleiro[], int jogadorAtual)
{
    if (tabuleiro[6] > 24)
    {
        printf("Parabéns! O Jogador A venceu com %d pedras no depósito.\n", tabuleiro[6]);
        return PLAYER; // Jogador A vence
    }
    else if (tabuleiro[13] > 24)
    {
        printf("Parabéns! O ");
        if (jogadorAtual == COMPUTER)
        {
            printf("computador");
        }
        else
        {
            printf("Jogador B");
        }
        printf(" venceu com %d pedras no depósito.\n", tabuleiro[13]);
        return COMPUTER;
    }
    return -1; // Sem vencedor ainda
}
```

*** `obterMovimentoComputador(int tabuleiro[])`**

- Propósito: Determinar a jogada do computador quando o mesmo for o adversário.
- Argumentos: Array que guarda o estado do tabuleiro ao longo do jogo.
- Valor de Retorno: Int.

```
int obterMovimentoComputador(int tabuleiro[])
{
    int attempt = 0;

    while (attempt < MAX_ATTEMPTS)
    {
        int movimento = rand() % 6; // Escolha aleatória entre 0 e 5

        if (tabuleiro[movimento] == 0)
        {
            attempt++;
            continue; // Evitar casas vazias
        }

        int simulacaoTabuleiro[TAM_TABULEIRO];
        for (int j = 0; j < TAM_TABULEIRO; ++j)
        {
            simulacaoTabuleiro[j] = tabuleiro[j];
        }

        int posicao = fazerMovimento(simulacaoTabuleiro, COMPUTER, movimento);

        // Verificar se o movimento é inválido
        if (simulacaoTabuleiro[posicao] == 1 && ((posicao >= 0 && posicao <= 5 && COMPUTER == PLA
        {
            attempt++;
            continue; // Tentar outro movimento
        }

        return movimento;
    }

    //Se o loop exceder o nº máximo de tentativas, escolhe um nº random válido
    int validMove = rand() % 6;
    while (tabuleiro[validMove] == 0)
    {
        validMove = rand() % 6;
    }

    return validMove;
}
```

Melhorias na Estrutura do Código

Para aprimorar a estrutura do código, considera-se a implementação de funções auxiliares para tarefas específicas, como movimentação de pedras, verificação de capturas e lógica do adversário virtual. Isso contribuiria para manter cada função mais concisa e facilitar a compreensão do código.

Conclusão

A implementação do jogo "Ouri" em C busca proporcionar uma experiência interativa fiel às regras do jogo. A estrutura modular facilita a manutenção e expansão do programa, permitindo melhorias futuras. O programa é executado via terminal, oferecendo opções para iniciar um novo jogo contra um Jogador ou IA. A interação com o jogador é intuitiva, e o programa exibe o vencedor ao final da partida.